

CSE 3251: Web Engineering

Credits	3
Contact Hours	42
Year	Third
Semester	Second
Prerequisites	CSE 2232: User Interface Development Lab · CSEXXXX: Computer Networks
Course Type	☒ Theory ☐ Laboratory work ☐ Project work ☐ Viva Voce

Motivation

Web applications are now a dominant software platform, serving users across devices and networks. Developing secure, scalable, and maintainable web systems requires a disciplined engineering approach. This course introduces that approach by combining foundational web engineering concepts with modern topics such as API design, security, scalability, containerisation, and CI/CD.

Course Objective

This course introduces Web Engineering as a systematic process for designing, analysing, and evaluating modern web applications. Students will study foundational web engineering concepts, REST API design, modern web architecture, web security, testing, scalability, containerisation, and CI/CD practices. The companion lab course (CSE 3252) provides hands-on implementation using Express.js, AdonisJS, and/or Laravel.

Course Outcomes (COs), Program Outcomes (POs) and Assessment

CO No.	CO Statement	Corresponding PO	Domain / Level of Learning Taxonomy	Delivery Methods & Activities	Assessment Tools
CO1	To analyse web engineering methodologies for the systematic development of web applications	Engineering Knowledge (PO1), Problem Analysis (PO2)	Cognitive domain – Level 4	☒ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Journal paper	☒ Class Test ☒ Final Exam ☒ Assignment ☐ Participation ☐ Presentation
CO2	To evaluate RESTful APIs and modern web architectures, including Single Page Applications and microservices	Design/Development of Solutions (PO3)	Cognitive domain – Level 5	☒ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☐ Journal paper	☒ Class Test ☒ Final Exam ☒ Assignment ☐ Participation ☐ Presentation
CO3	To analyse and evaluate web application security threats and practical defences, including injection attacks, authentication flaws, information leaks, and denial of service attacks	Investigation (PO4)	Cognitive domain – Level 5	☒ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☐ Journal paper	☒ Class Test ☒ Final Exam ☒ Assignment ☐ Participation ☐ Presentation
CO4	To explain and compare scalability strategies, Docker-based containerisation, and CI/CD pipeline concepts for	Modern Tool Usage (PO5)	Cognitive domain – Level 4	☒ Lecture Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Journal paper	☒ Class Test ☒ Final Exam ☒ Assignment ☐ Participation ☐ Presentation

CO No.	CO Statement	Corresponding PO	Domain / Level of Learning Taxonomy	Delivery Methods & Activities	Assessment Tools
	production web systems				

Assessment and Marks Distribution

Students will be assessed on the basis of their overall performance across all class tests, assignments, and the final examination.

Component	Weightage
Class Tests + Assignments (distributed throughout the semester)	20%
Comprehensive Final Exam (3 hours)	70%
Class Participation	10%

Course Contents

Week 1 — Web-Based Systems and Web Engineering

- Characteristics of web-based systems: network intensity, concurrency, unpredictability of load, performance sensitivity, content-driven nature, continuous evolution
- Categories of web applications: informational, interactive, transactional, workflow, collaborative, portal, and service-oriented systems
- Why Web Engineering is needed: differences from traditional software engineering
- Web Engineering layers: process, methods, tools, quality focus
- Course orientation: how theory supports the companion implementation lab

Week 2 — Web Engineering Process

- Process models for web development: incremental, agile-web, spiral, and iterative delivery

- Framework activities in web projects: communication, planning, modelling, construction, deployment
- Process adaptation for small teams vs. enterprise teams
- Risk in web projects: volatile requirements, evolving content, rapid release pressure
- Quality attributes in the process: usability, security, maintainability, scalability

Week 3 — Communication and Planning

- Stakeholder communication in web projects: clients, content owners, developers, end users
- Requirement elicitation techniques: interviews, workshops, scenarios, user stories
- Scope definition and feasibility analysis for a web application
- Project planning: scheduling, effort estimation, milestones, deliverables
- Team roles, project tracking, and essential metrics for web projects

Week 4 — Modelling Activity

- Analysis modelling for web applications: information flow, functional needs, user interaction
- Content modelling: content objects, relationships, metadata
- Interaction modelling: user scenarios, navigation paths, use-case-driven behaviour
- Architectural modelling: layering, components, separation of concerns
- Validation of models before implementation

Week 5 — Web App Design: Interface, Content, Architecture, and Navigation

- Design issues specific to web apps: aesthetics vs. function, cross-browser consistency
- Interface design: interaction design principles, affordance, feedback
- Typography, layout, colour, and visual hierarchy
- Content design: information architecture, labelling, metadata
- Architecture and navigation design: MVC, layered architecture, navigation semantics and structure
- Object-Oriented Hypermedia Design (OOHDM) and design metrics for web apps

Week 6 — REST API Design and Web Services

- HTTP as an application protocol: methods (GET, POST, PUT, PATCH, DELETE), status codes, headers
- REST architectural constraints: statelessness, uniform interface, resource identification, HATEOAS
- Designing RESTful APIs: resource naming conventions, URI structure, versioning strategies
- API data formats: JSON vs. XML; JSON Schema for validation
- API documentation standards: OpenAPI / Swagger specification
- Web services: SOAP vs. REST comparison; GraphQL as a query-based alternative
- API security fundamentals: API keys, OAuth 2.0 overview, JWT structure

Week 7 — Modern Web Architecture

- Single Page Applications (SPAs): architecture, client-side routing, state management
- Server-Side Rendering (SSR) vs. Client-Side Rendering (CSR) vs. Static Site Generation (SSG): trade-offs
- Introduction to microservices: monolith vs. microservice architecture, inter-service communication (REST, message queues)
- Server-side framework patterns — MVC in Express.js, AdonisJS, and Laravel: routing, middleware, ORM (conceptual overview; implementation in CSE 3252)
- Backend-as-a-Service (BaaS) and serverless concepts: AWS Lambda, Firebase

Week 8 — Web App Security I: Core Application Attacks

Primary text: Malcolm McDonald, *Web Security for Developers: Real Threats, Practical Defense*

- OWASP Top 10 overview and the role of application security in Web Engineering
- **Injection attacks:** input handling failures, SQL Injection, command/query injection, and secure coding practices such as parameterised queries and ORM use
- **Cross-Site Scripting (XSS):** reflected, stored, and DOM-based XSS; attacker goals; output encoding and safe rendering practices
- **Cross-Site Request Forgery (CSRF):** browser trust model, forged requests, CSRF tokens, and cookie-based protections
- Secure input handling, validation, sanitisation, and output encoding as general defensive principles

Week 9 — Web App Security II: Authentication, Information Exposure, and Availability

Primary text: Malcolm McDonald, *Web Security for Developers: Real Threats, Practical Defense*

- **Compromising authentication:** weak passwords, brute-force attacks, credential stuffing, broken login flows, and authentication hardening principles
- **Session and cookie security:** session identifiers, hijacking/fixation concepts, and the use of `HttpOnly`, `Secure`, and `SameSite`
- **Information leaks:** verbose error messages, debug endpoints, stack traces, sensitive metadata, exposed configuration, and unintended data disclosure
- **Denial of Service (DoS) attacks:** request flooding, resource exhaustion, expensive operations, abusive endpoints, and rate limiting / throttling concepts
- Security checklist for web application deployment and maintenance

Week 10 — Web App Testing

- Testing web apps vs. traditional software: challenges of dynamic content, browser variance, network dependency
- **Content testing:** verifying correctness, completeness, and consistency of content objects
- **User interface testing:** navigational testing, form testing, usability evaluation
- **Navigation testing:** link validation, navigational flow, dead-end detection
- **Configuration testing:** cross-browser, cross-device, cross-OS compatibility testing
- **Security testing:** penetration testing concepts, OWASP ZAP overview, automated vulnerability scanning
- **Performance testing:** load testing, stress testing, baseline and benchmark metrics; tools: JMeter, k6
- Test automation strategies for web apps: Selenium, Playwright, Cypress (conceptual overview)

Week 11 — Web App Scalability I: Concepts and Strategies

- What scalability means: cost-effective adjustment to demand (users, data, interaction rate)

- Root causes of scalability issues: concurrency, data volume, high interaction rates
- **Vertical scaling (scale-up):** adding CPU, RAM, SSD — benefits, limits, single point of failure
- **Horizontal scaling (scale-out):** multiple instances behind a load balancer — benefits and complexity
- **CDN (Content Delivery Networks):** static asset delivery, geographic distribution, bandwidth offloading; providers (Cloudflare, AWS CloudFront)
- **Caching strategies:** client-side caching (HTTP Cache-Control headers), server-side caching (Redis, Memcached), cache invalidation
- **Database scaling:** read replicas, sharding (horizontal partitioning), partitioning strategies
- **Asynchronous processing:** message queues (RabbitMQ, AWS SQS), pub/sub patterns, decoupling long-running tasks

Week 12 — Web App Scalability II: Load Balancing

- Load balancer role in the system architecture: request flow, health checks, server pool management
- **Load balancing algorithms:**
 - Round Robin: equal distribution, limitations with session state
 - Weighted Round Robin: capacity-aware distribution
 - Least Connections: dynamic load-aware routing
 - IP Hash: client-affinity routing
- **Session persistence (sticky sessions):** when required, trade-offs with scalability
- **SSL/TLS termination at the load balancer:** offloading decryption, centralised certificate management
- **GeoDNS:** mapping domains to geographically nearest servers; latency reduction
- Layer 4 vs. Layer 7 load balancing: transport-layer vs. application-layer routing
- Real-world load balancer examples: Nginx, HAProxy, AWS ALB/NLB

Week 13 — Containerisation with Docker

- Virtual machines vs. containers: isolation model, resource overhead, startup time comparison
- **Docker architecture:** Docker Engine, images, containers, Docker Hub registry, Dockerfile

- Writing a Dockerfile: `FROM`, `COPY`, `RUN`, `EXPOSE`, `CMD` — anatomy and best practices
- Building and running images: `docker build`, `docker run`, port mapping, environment variables
- **Docker Compose**: defining multi-container applications (web app + database + cache); `docker-compose.yml` structure; `up`, `down`, `logs`
- Container networking: bridge networks, service discovery by container name
- **Container orchestration concepts**: why Kubernetes exists; nodes, pods, deployments, services — conceptual overview (hands-on in advanced courses)
- Docker in the development workflow: consistent environments, "works on my machine" problem solved

Week 14 — CI/CD Concepts and DevOps Practices

- What is CI/CD? Continuous Integration, Continuous Delivery, Continuous Deployment — definitions and differences
- **CI pipeline stages**: source (Git trigger) → build → automated test → lint/static analysis → artefact creation
- **CD pipeline stages**: artefact → staging deploy → integration tests → production deploy
- **GitHub Actions**: workflow YAML structure, triggers (`push`, `pull_request`), jobs, steps, actions marketplace
- Containerised CI/CD: building Docker images in a pipeline, pushing to a registry (Docker Hub, GitHub Container Registry)
- Deployment strategies: blue-green deployment, canary releases, rolling updates — concepts and trade-offs
- Environment management: development, staging, production; environment variables and secrets management
- DevOps culture: shared responsibility, observability basics (logs, metrics, alerts)

Textbooks

1. Roger S. Pressman and David Lowe — *Web Engineering*, Tata McGraw-Hill, 2008 — for foundational topics on web-based systems, web engineering process, communication, planning, and modelling activity

2. Malcolm McDonald — *Web Security for Developers: Real Threats, Practical Defense*, No Starch Press, 2020
3. Leonard Richardson & Mike Amundsen — *RESTful Web APIs*, O'Reilly Media, 2013
4. Nigel Poulton — *Docker Deep Dive* (2023 edition) (free online at dockerbook.com)

Recommended Resources

Scalability

1. **Hookdeck — Web Scalability for Beginners** — hookdeck.com/blog/web-scalability-beginners — conceptual overview of scaling strategies
2. **Design Gurus — Introduction to Load Balancing** — [designgurus.io — Grokking System Design Fundamentals](https://designgurus.io/Grokking-System-Design-Fundamentals) — load balancer mechanics and algorithms

Security

3. **OWASP Top 10** — owasp.org/www-project-top-ten — authoritative web security threat list
4. **OWASP Cheat Sheet Series** — cheatsheetseries.owasp.org — practical defence guidance per vulnerability

REST & Architecture

5. **MDN — HTTP documentation** — developer.mozilla.org/en-US/docs/Web/HTTP
6. **OpenAPI Specification** — spec.openapis.org — API documentation standard

CI/CD

7. **GitHub Actions Documentation** — docs.github.com/en/actions
8. **Atlassian CI/CD Guide** — atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment

Free PDF Quick-Reference Books (goalkicker.com)

Topic	Download
JavaScript / HTTP Requests	JavaScript Notes for Professionals (PDF)

Topic	Download
Git	Git Notes for Professionals (PDF)
Linux (for Docker context)	Linux Notes for Professionals (PDF)
Node.js	Node.js Notes for Professionals (PDF)